Legal Contract Review

Technical documentation set

20 documents

Documentation set: Papyrus, Legal Contract Review

Version 1, 2026-07-28. Written against the code as it stands, not as planned.

#	Document	What it answers
01	Requirements	What the system must do, numbered and testable
02	High-Level Design	The components, the data flow, the tech choices
03	Low-Level Design	Inside each component: nodes, agents, state, error handling
04	Data Model	Tables, the clause and finding shapes, the precedent collection
05	API Specification	Every endpoint, its inputs, outputs, and status codes
06	Decision Log (ADRs)	Each architectural decision, its options and consequences
07	Configuration and Secrets	Every environment variable and where its real value lives
08	Runbook	Start, stop, seed, recover, and what to do when it breaks
09	Test Plan and Report	What is tested, how, and the last measured result
10	Benchmark Report	Measured recall, attribution, severity and latency on a planted corpus
11	Non-Functional Requirements	Performance, availability, privacy targets with numbers
12	Security Review	Threat model: what an attacker could try and what stops them
13	Privacy and Data Handling	What confidential data is touched and where it goes
14	Model Card	Which model, its limits, known failure modes
15	Risk Register	What could sink the project and what mitigates it
16	Traceability Matrix	Requirement to code to test, so coverage is provable
17	User Guide	How a lawyer operates the system
18	Release Notes	What changed, in order, and what it broke
19	Handover	Everything the next owner needs

Also in this folder: the client requirement PDF, and demo-script.md for the walkthrough.

01. Requirements

Version 1, 2026-07-28. Authoritative source: the client requirement PDF in this folder, plus the project scope and design documents in the planning repository. Status is measured against the code, and the evidence is in 16-traceability-matrix.md.

1. Problem

Reviewing a commercial contract means reading every clause, comparing it against the firm's standard wording and rules, spotting what is missing as well as what is wrong, drafting replacement language, and explaining to a client what each issue actually means. It is slow, it is expensive, and the quality depends on which lawyer picked up the file.

2. What this product is

A multi-agent assistant **for a lawyer**. It reads an uploaded contract, splits it into numbered clauses, inspects every clause four ways in parallel, drafts replacement wording for what it flags, rolls the whole thing into a risk level and a plain-English summary, and hands it to a lawyer who accepts, rejects or edits **clause by clause**. Nothing is signed off without a human, and the corrected document can be exported with only the changed clauses rewritten.

Contracts never touch a cloud model. There is exactly one self-hosted model lane.

3. Functional requirements

ID	Requirement	Status
FR-1	Accept a contract as an uploaded file, normalise it to plain text, and identify its type, parties and key dates	MET, `.docx` and `.pdf`
FR-2	Split the contract into numbered clauses with headings, original wording, and a clause type from a fixed vocabulary	MET
FR-3	Inspect every clause for rule breaches, commercial risk, deviation from the firm's template, and financial terms	MET, four inspectors running in parallel
FR-4	Identify required clauses that are **absent** from the contract, not only faults in what is present	MET
FR-5	Every finding must carry a clause reference, an inspector, a severity, a plain-English line, the legal term, what is wrong, what to change, what happens if it is ignored, and a quote of the offending wording	MET, enforced by validation; incomplete findings are dropped and counted
FR-6	Draft replacement wording for flagged clauses, with an ask, a fallback and a walk-away position	MET
FR-7	Roll findings into a contract-level risk level and score, and an executive summary in plain English	MET

ID	Requirement	Status
FR-8	Let a lawyer accept, reject or edit each flagged clause, and record which	MET
FR-9	Retrieve comparable prior reviews as precedent, so later contracts can cite earlier rulings	MET
FR-10	Keep a tamper-evident audit trail of every pipeline step, readable through the API	MET, and verified against a forged database entry
FR-11	Export the reviewed contract as the original document with only the changed clauses rewritten	MET for `.docx`; PDF is refused rather than silently mangled
FR-12	Administer users: an administrator creates lawyer accounts; there is no open signup	MET

4. Non-functional requirements

ID	Requirement	Target	Status
NFR-1	Contract text never reaches a third-party model	Zero exceptions	MET by construction: one lane, no cloud client in the codebase
NFR-2	A whole contract review completes without human intervention	under 45 minutes	MET, measured mean 429 seconds over 13 contracts
NFR-3	A stuck or crashed agent must not hang the review	Per-node guard	MET, guarded nodes with a 1200 second ceiling and a retry
NFR-4	Parallel inspectors must not multiply GPU cost	One container	MET, single-container lane plus a client-side lock that serialises calls. A vLLM swap briefly removed the lock on 2026-07-28; it was rolled back on 2026-07-29 for measured recall loss (ADR-014)
NFR-5	A half-formed finding is never shown to a lawyer	Validation before display	MET
NFR-6	The audit trail detects tampering	Detection, not prevention	MET
NFR-7	Synthetic contracts only, secrets outside the repository	No real client data	MET
NFR-8	Reuse the skeleton from the ticket-triage system	Module-level reuse	MET

5. Out of scope

No scanned-document handling beyond text extraction, no optical character recognition, no signature or execution workflow, no matter management or billing, no jurisdiction-specific legal advice, and no automatic sending of anything to a counterparty. The system produces a marked-up document for a lawyer; the lawyer decides what leaves the building.

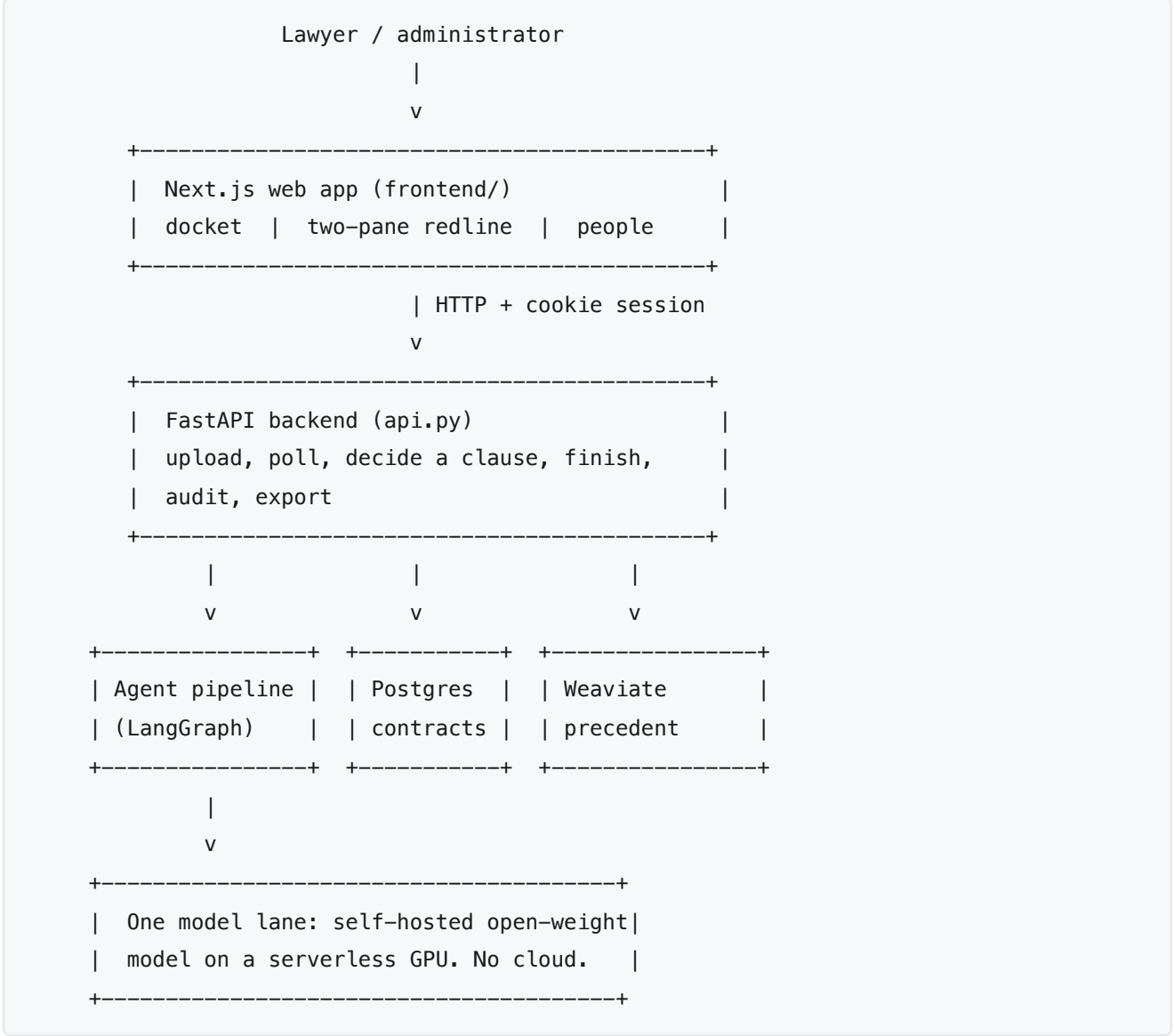
6. Known requirement gaps

- Attribution is imperfect: a planted defect is often caught by a different inspector than the one that should have caught it. Recall is what matters and it is measured; attribution is reported honestly at 71 percent.
- A contract with no findings is reported as low risk, which is indistinguishable from a slightly untidy one. This is pinned by a test so that changing it is a deliberate act.

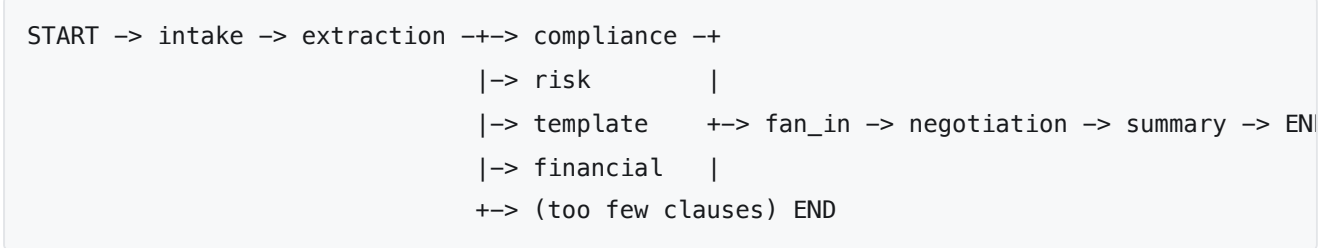
02. High-Level Design

Version 1, 2026-07-28.

1. The shape of the system



2. The pipeline



Nine nodes. The four inspectors run as one parallel step and write only to append-only keys, so their results cannot overwrite each other. Fan-in is plain code: it pins each finding to its clause, drops invalid ones, and rolls up the contract risk.

Node	Model call	Responsibility
`intake`	no	File to plain text, plus the document's format
`extraction`	yes	Clause list with numbers, headings, wording and types, plus parties, contract type and dates
`compliance`	yes, with tools	Breaches of the firm's rules pack
`risk`	yes, with tools	Liability caps, one-way indemnities, intellectual-property grabs, auto-renewal, restraint
`template`	yes, with tools	Deviations from the standard, and required clauses that are missing
`financial`	yes, with tools	Payment days, fee increases, interest, penalties, totals that do not add up
`fan_in`	no	Attach findings to clauses, drop invalid ones, roll up risk
`negotiation`	yes	One replacement proposal per flagged clause, with ask, fallback and walk-away
`summary`	yes	Executive summary and counts

3. Components

Component	Where	Responsibility
Web app	`frontend/`	Docket with live stage narration, two-pane redline review, people administration
API	`api.py`	Upload, polling, clause decisions, finish, audit read, export
Pipeline	`app/graph.py`	The nine nodes and their guards
Agents	`app/agents.py`, `app/agents_base.py`	Prompts, the reasoning loop, finding validation
Tools	`app/tools.py`	Template fetch, rules read, precedent search
Model lane	`app/router.py`	One endpoint, one lock, one retry
System of record	`app/store.py` on Postgres	One row per contract, plus the uploaded bytes
Precedent cabinet	`app/precedent.py` on Weaviate	Finished reviews, retrieved by similarity
Export	`app/export.py`	Rewrites the original document in place

Component	Where	Responsibility
Audit	`app/audit.py`	Hash chain over every step

4. Flow of one review

1. A lawyer uploads a file. The API stores the bytes, creates a row, and returns immediately.
2. The pipeline runs in the background. The docket row narrates the stage: reading, extracting, inspecting, negotiating, summarising, done.
3. The finished review shows every clause, its findings, and a proposed replacement where there is one.
4. The lawyer accepts, rejects or edits each flagged clause. The final wording is whichever they chose.
5. Finishing the review files it into the precedent cabinet, so a later contract can retrieve it.
6. The corrected document can be exported: the original file with only the changed clauses rewritten.

5. Technology choices

Layer	Choice	Reason
Backend	Python and FastAPI	Shared skeleton with the ticket-triage system
Agent wiring	LangGraph	Parallel fan-out with append-only merge keys is exactly the shape needed
Vector store	Weaviate	Precedent search by meaning, running locally
Relational store	Postgres	One row per contract, whole state as JSON, plus the original file bytes
Frontend	Next.js, TypeScript, Tailwind	Shared skeleton
Model	One self-hosted open-weight model on serverless GPU	Contracts are confidential; there is no cloud lane at all

6. What is deliberately different from the ticket-triage system

- **One lane, not four.** No cloud model, no tier switch, no routing grid. Confidentiality is absolute here, so the routing question does not arise.
- **Parallel inspectors, not a chain.** Four opinions on the same clause list, merged by plain code.
- **Clause-level human control.** The human gate is per clause rather than per document.
- **The document itself is the deliverable**, so the original bytes are kept and rewritten on export.

03. Low-Level Design

Version 1, 2026-07-28.

1. The state object

One dictionary flows through the pipeline. Two keys are append-only reducers, which is what makes the parallel inspector step safe.

Key	Written by	Holds
`contract_id`, `filename`, `source_format`	intake	Identity and the document format
`status`	several	`processing`, `extraction_failed`, `needs_review`, `reviewed`, `error`
`stage`	every node	Narration key the docket polls: reading, extracting, inspecting, negotiating, summarising, done
`meta`	extraction	Parties, contract type, key dates, pages
`raw_text`	intake	Normalised plain text
`clauses`	extraction, then fan-in	The clause list, each with findings, a proposal, a decision and final wording
`findings_raw`	four inspectors, append-only	Every finding before validation
`inspector_reports`	four inspectors, append-only	Per inspector: ok or failed, plus a note
`missing_clauses`	template inspector	Required clauses absent from the document
`contract_risk`	fan-in	Level, score and a one-line why
`negotiation_points`	negotiation	Ask, fallback and walk-away per clause
`summary`	summary	Executive text and counts
`audit`	every node	The hash chain, append-only

2. The three shapes

Clause: a stable id, the printed number, heading, original wording, a type from a fixed vocabulary of fourteen, its findings, at most one proposal, the lawyer's decision, and the final text.

Finding: a finding id, the clause it belongs to, which inspector raised it, a severity, a plain-English line, the legal term, what is wrong, what to change, what happens if it is ignored, a quote of the offending wording, and an optional hint for the fix.

Proposal: the clause it replaces, the full replacement wording, the span struck and the span inserted for the redline, and the finding ids it answers.

A finding is discarded unless **all nine required fields are present** and the severity is one of high, medium or low. Half-formed findings are counted, never shown.

3. The reasoning loop

Each inspector is a ReAct agent: it thinks, may call a tool, reads the result, and repeats, up to six steps, after which it must produce its findings. Three details matter:

- **Parsing takes the first complete JSON object** from the model's output, using a streaming decoder. The earlier version sliced from the first brace to the last, which broke deterministically whenever the model emitted a second object after its answer.
- **A generous token ceiling per call.** A finding carries nine fields, so several findings in one answer will exceed a small limit and be cut off mid-string. Truncation shows up as a parse failure at the identical character on both attempts, which is the fingerprint to recognise.
- **Unknown tool names are corrected** with the real options rather than failing the step.

The three tools are read-only: fetch the firm's template for a contract type, read the rules pack, and search the precedent cabinet. All three resolve their paths relative to the code, not the working directory, so launching the server from elsewhere cannot silently tell every inspector that the rules do not exist.

4. Guards and failure handling

Every node is wrapped in a guard that gives it a bounded wall-clock and one retry, then records the failure in the state rather than raising into the framework.

Failure	Behaviour
Fewer than three clauses extracted	The review stops with `extraction_failed`; a document that could not be read is never inspected
One inspector fails both attempts	Its report is marked failed with a note; the other three still produce findings and the review continues
Negotiation or summary fails	The review still reaches the lawyer with findings; the missing part is visible
The model lane returns a stray server error	One retry, because a container swap mid-run surfaces exactly this way
Parallel calls arriving together	A client-side lock serialises them, and the platform-side single-container cap prevents a second billed GPU. A vLLM swap removed the lock on 2026-07-28 and was rolled back on 2026-07-29 (ADR-014)

5. Fan-in, in plain code

Fan-in is deliberately not an agent. It pins each valid finding to its clause by id, separates an incomplete finding from one naming a clause that does not exist, counts both, rolls severities into a contract risk level and a score, and normalises the capitalisation the model sometimes returns. Doing this in code rather than in a prompt is what makes the numbers reproducible.

6. Export

Export rewrites the **original** document rather than generating a new one: it matches each changed clause's original wording against a moving window of paragraphs, keeps the first paragraph's style, and collapses the rest of the matched span. Clauses it cannot locate are named in a response header and surfaced in the interface as a fix-by-hand note. A PDF upload is refused with a clear status rather than mangled, and a contract uploaded before the feature existed is refused with a re-upload instruction.

7. Module map

Module	Responsibility
`api.py`	Endpoints, authentication, background processing
`app/graph.py`	Nine nodes, guards, conditional edge after extraction
`app/agents.py`	Six agent prompts plus finding stamping and inspector running
`app/agents_base.py`	The reasoning loop and the JSON parser
`app/tools.py`	The three read-only tools and the registry
`app/router.py`	The single model lane, retry, timeout
`app/store.py`	Postgres access
`app/precedent.py`	Weaviate collection, lazy embedding load
`app/export.py`	Document rewrite
`app/intake.py`	File to text
`app/state.py`	Shapes, validation, risk roll-up
`app/audit.py`	Hash chain
`app/users.py`	Accounts, roles, sessions
`app/report.py`	The printable review report

04. Data Model

Version 1, 2026-07-28. Two stores: Postgres holds one row per contract, Weaviate holds finished reviews as searchable precedent.

1. Postgres, database `contracts`

Created at startup by an idempotent initialiser. New columns are added in the same place with `ADD COLUMN IF NOT EXISTS`, so an existing database migrates itself on boot.

`contracts`

Column	Type	Meaning
`contract_id`	TEXT, primary key	Identity of the review
`filename`	TEXT	The uploaded file's name
`status`	TEXT	`processing`, `extraction_failed`, `needs_review`, `reviewed`, `error`
`stage`	TEXT	Narration for the docket: reading, extracting, inspecting, negotiating, summarising, done
`risk_level`	TEXT	`high`, `medium`, `low`, rolled up from the findings
`state`	JSONB	The entire pipeline state: clauses, findings, proposals, decisions, summary, audit chain
`created_at`	TIMESTAMPTZ	Upload time
`file_bytes`	BYTEA	The original uploaded document, kept so export can rewrite it in place

Status, stage and risk are duplicated out of the state as real columns because the docket lists and filters on them; everything else is read from the JSON.

Accounts

Managed by the authentication library: id, email, hashed password, active flag, and a `role` of `lawyer` or `admin`. **There is no open signup and no customer role.** An administrator creates every account.

2. The shapes inside `state`

Clause

Field	Meaning
`clause_id`	Stable id assigned at extraction, for example `c04`

Field	Meaning
`number`, `heading`	As printed in the document
`text`	The original wording
`clause_type`	One of fourteen: parties, scope, payment, term and termination, confidentiality, intellectual property, liability, indemnity, restraint, data protection, governing law, notices, boilerplate, other
`findings`	The findings pinned to this clause at fan-in
`proposal`	At most one replacement, authored by the negotiation agent
`decision`	`accepted`, `rejected`, `edited`, or nothing yet
`final_text`	The original wording, the proposal, or the lawyer's own edit

Finding

Nine fields are required, and a finding missing any of them is discarded before a lawyer ever sees it: the clause it belongs to, which inspector raised it, severity, a plain-English line, the legal term, what is wrong, what to change, what happens if it is ignored, and a quote of the offending wording. An optional hint feeds the negotiation agent.

The plain-English line comes first by design: the product's claim is that a non-lawyer can understand why a clause is a problem.

Proposal

The clause it replaces, the full replacement wording, the struck span and the inserted span for the redline view, and the finding ids it answers.

Missing clause

A required clause that is absent from the document, with a severity. Absence is treated as a first-class finding rather than a footnote.

3. Weaviate, collection `Precedent`

Property	Meaning
Title and summary of a prior review	What the search returns
Contract type	For filtering by like-for-like
`source`	`seed` for the cold-start entries, otherwise a real filed review

Search has a relevance floor, and embeddings are computed locally so no contract text is sent anywhere to be indexed.

Integrity point, deliberately preserved: the seeded precedent entries are *not* built from the benchmark contracts. If they were, an inspector could retrieve a planted defect instead of finding it, and the recall number in 10-benchmark-report.md would measure nothing. The seeds cover the same classes of term with different counterparties.

4. Audit chain

Every pipeline step appends an entry holding the step, the previous entry's hash, and its own. The chain is stored inside the state and read back through the audit endpoint, which verifies it on read and reports the first broken index. This has been tested against a forged row written directly into Postgres, and the break was reported at the exact index.

5. Retention

Contracts, their state, and the original bytes are kept indefinitely. Finished reviews are additionally copied into the precedent cabinet. There is no purge, no retention schedule, and no deletion workflow. All contracts in the system are synthetic.

6. Backup and recovery

Both stores are Docker volumes on the developer machine. Recovery means re-running the seed scripts: accounts, and the precedent cold start. There is no scheduled backup, which is acceptable only because the data is synthetic and reproducible.

05. API Specification

Version 1, 2026-07-28. Base URL `http://localhost:8000`. The framework serves a live version at `/docs`; this is the reviewed narrative version.

1. Authentication and roles

Sessions are a signed cookie. Two roles: `lawyer` and `admin`. **There is no open signup**; an administrator creates every account. Every endpoint below except health, configuration and the two first-run setup routes requires a signed-in account.

The one exception to "an administrator creates every account" is the **first-run bootstrap**. A freshly installed system has no accounts, therefore no administrator, therefore no way to create the first one. `GET /auth/needs-setup` reports that state and `POST /auth/bootstrap` closes it by creating the founding administrator. The bootstrap route refuses with 403 the moment any account exists, so the door opens once and never again.

Method	Path	Who	Notes
POST	`/auth/login`	anyone	Form credentials (email address), returns a session cookie
POST	`/auth/login-flex`	anyone	Same, but the identifier may be **either** the email address or the username, matched case-blind. This is what the sign-in screen posts to
GET	`/auth/needs-setup`	anyone	`{"needs_setup": true}` only while the system holds zero accounts
POST	`/auth/bootstrap`	anyone, once	Creates the founding administrator from email, username and password. 403 once any account exists, 409 on a duplicate address
POST	`/auth/logout`	signed in	Ends the session
GET	`/users/me`	signed in	The current account and its role

2. Public

Method	Path	Returns
GET	`/`	Liveness. Answers instantly without touching anything, so it is safe for uptime pings
GET	`/healthz`	The honest check: opens Postgres and Weaviate and reports each as `ok` or `down`, with `status` `ok` or `degraded`. Reports up or down only, never the error text, because the route is unauthenticated and driver errors leak host and user strings
GET	`/config`	Brand name and tagline

3. Contracts

Method	Path	Who	Notes
POST	`/contracts`	lawyer or admin	Multipart upload of a `.docx` or `.pdf`. Returns immediately with an id while the pipeline runs in the background. 400 on an empty file
GET	`/contracts`	lawyer or admin	The docket: id, filename, status, stage, risk level, created time
GET	`/contracts/{id}`	lawyer or admin	The full review: clauses, findings, proposals, decisions, summary. 404 if unknown
GET	`/contracts/{id}/audit`	lawyer or admin	The hash chain with a verification result. 404 if unknown
GET	`/contracts/{id}/report`	lawyer or admin	The printable review report

The docket is polled while a review runs; the `stage` field is what the row narrates.

4. Clause decisions

Method	Path	Body	Notes
POST	`/contracts/{id}/clauses/{clause_id}/decision`	`{verdict, edited_text?}`	`verdict` is `accepted`, `rejected` or `edited`

Status codes, all of them deliberate:

Code	When
422	The verdict is not one of the three
422	`edited` was sent with no replacement wording
422	`accepted` was sent for a clause that has no proposal to accept
404	Unknown contract, or unknown clause
409	The review is already finished

The final wording is set from the verdict: accepting takes the proposal, rejecting keeps the original, editing takes the lawyer's text.

5. Finishing a review

Method	Path	Notes
POST	`/contracts/{id}/finish`	Locks the review and files it into the precedent cabinet

Code	When
404	Unknown contract
409	Already finished
409	The review is not ready to finish
409	Some flagged clause has no decision yet

A precedent-store outage must never block a lawyer signing off; that is covered by a test.

6. Export

Method	Path	Returns
GET	`/contracts/{id}/export`	The original document with only the changed clauses rewritten

Code	When
409	The review is not finished yet
422	The upload was a PDF; export is `.docx` only
410	The contract was uploaded before the original bytes were kept: upload it again

Clauses whose original wording could not be located in the document are named in a response header, and the interface shows a fix-by-hand note. The limitation is surfaced rather than silent.

7. User administration

Method	Path	Who	Notes
The People screen is the administrator's landing page after sign-in and is the only place accounts are made. An administrator can create administrators as well as lawyers.			

Method	Path	Who	Notes
GET	`/users`	admin	All accounts
POST	`/users`	admin	Create an account from email, username, password and role. 422 if the role is not lawyer or admin, 409 if either the address **or the username** already exists
PATCH	`/users/{id}`	admin	Change role, address, username or password. 404 unknown, 409 duplicate address or username, 422 bad role
DELETE	`/users/{id}`	admin	Deactivate. 400 if you attempt it on your own account

8. Status codes used

Code	Meaning here
200	Success
400	Empty upload, or deactivating yourself
401	No session
403	Wrong role
404	Unknown contract, clause or account
409	Already finished, not ready, undecided clauses, duplicate account
410	The original file was never stored
422	Invalid verdict, missing edited wording, nothing to accept, wrong file type for export, invalid role

06. Decision Log (Architecture Decision Records)

Version 1, 2026-07-28.

ADR-001. Fork the ticket-triage skeleton rather than start fresh

Context. A second multi-agent system was needed in days, not weeks, and a working skeleton already existed. **Options.** Build from scratch; extract a shared library first; copy the repository and adapt. **Decision.** Copy-fork, strip the domain, swap the authentication model, keep the pipeline shape. **Consequences.** The system was standing in days. The cost is real duplication, since several modules now exist in more than one repository, which has already caused a bug fixed twice. The plan to replace copies with a shared package is recorded in the planning repository.

ADR-002. One model lane, no cloud, no tier switch

Context. Contracts are confidential by nature. The ticket system's routing grid exists because some tickets are safe to send to a provider; here none are. **Decision.** Exactly one lane, a self-hosted open-weight model on a serverless GPU. No cloud client, no key, no switch. **Consequences.** The privacy claim is provable by reading the code rather than by trusting a rule. The cost is that model quality is capped by what fits on a rented GPU, and every measurement run costs money.

ADR-003. Four inspectors in parallel, merged by plain code

Context. A clause can be wrong in unrelated ways: it can breach a rule, carry commercial risk, deviate from the standard, or be financially wrong. **Options.** One agent with a long prompt; four agents in sequence; four in parallel. **Decision.** Four in parallel, each writing only to append-only keys, merged by a plain-code fan-in that pins findings to clauses, drops invalid ones and rolls up risk. **Consequences.** Each inspector has a short, sharp prompt, and the merge is reproducible because it is not a model. The cost is that attribution blurs: an inspector often catches a defect belonging to another's beat, which is why the benchmark reports recall and attribution separately.

ADR-004. A finding is worthless unless it is complete

Context. Early output produced fragments: a severity with no evidence, a complaint with no fix. **Decision.** Nine fields are required, including a plain-English line, a quote of the offending wording, and what happens if it is ignored. A finding missing any of them is dropped and counted. **Consequences.** What a lawyer sees is always actionable. The cost is fewer findings, and a stricter dependency on model obedience.

ADR-005. Absence is a finding

Context. The dangerous defect in a contract is often a clause that is simply not there. **Decision.** The template inspector reports required clauses that are missing, and they roll into risk exactly like faults in present clauses. **Consequences.** The benchmark tracks them separately and all three planted omissions were caught.

ADR-006. The human gate is per clause, not per document

Context. A contract review is not one decision; it is dozens. **Decision.** Every flagged clause needs an explicit accept, reject or edit, and the review cannot be finished until they all have one. **Consequences.** Sign-off is meaningful and traceable. The cost is more clicks, which is the correct trade for legal work.

ADR-007. Export rewrites the original document

Context. A lawyer's deliverable is the document, not a web page. **Options.** Generate a fresh document from the clause list; produce a change list; rewrite the original in place. **Decision.** Rewrite the original, matching each changed clause against a moving window of paragraphs, keeping the first paragraph's style. **Consequences.** Formatting, numbering and everything untouched survive. The limits are real and surfaced rather than hidden: `.docx` only, and any clause that cannot be located is named for manual fixing.

ADR-008. Precedent seeds must not come from the benchmark corpus

Context. The precedent cabinet is empty on a fresh machine, so early contracts get nothing from it. **Decision.** Seed eight fictional prior reviews covering the same classes of term as the benchmark, with different counterparties, tagged as seeds. **Consequences.** Cold start is solved without letting an inspector retrieve a planted defect instead of finding it, which would have made the recall number meaningless.

ADR-009. Pin the lane to a single container and serialise calls

Context. Four inspectors fan out at once. The platform responded by starting a second billed GPU, whose model was still loading, so requests queued for minutes and then failed. **Decision.** Cap the lane at one container, add a client-side lock so parallel nodes queue, and retry once on a stray server error. **Consequences.** Predictable cost and no mid-run container swaps. The cost is that the four inspectors run one after another in practice, which is most of the wall-clock time per contract.

ADR-010. Take the first complete JSON object from model output

Context. Parsing from the first brace to the last broke deterministically whenever the model emitted a second object after its answer. **Decision.** Decode the first complete object with a streaming decoder. **Consequences.** A whole class of deterministic failure disappeared. The same bug existed in the sibling governance system and was fixed there in the same session.

ADR-011. Do not change the risk roll-up before collecting numbers

Context. A contract with zero findings is reported as low risk, which is indistinguishable from a slightly untidy one. **Decision.** Leave it, and pin today's behaviour with a test, rather than change risk semantics immediately before a measurement run. **Consequences.** The benchmark numbers are comparable. The oddity is documented, and changing it later will break the test loudly rather than silently.

ADR-012. Roll back the vLLM + AWQ lane, keep the tolerant JSON parser

Context. On 2026-07-28 the lane moved from one-at-a-time transformers generation with a bitsandbytes-quantised model to vLLM serving the official AWQ checkpoint, batching up to 8 requests in one container. It was 3.3x faster on a single contract, but that same contract scored recall 3/5 against the earlier run's 5/5. One contract could not separate quantisation damage from run noise, so the decision was deferred and the headline benchmark numbers in these documents still described the old stack.

What was measured, 2026-07-29. The full 13-contract benchmark was rerun on vLLM, after fixing a parser bug found on the way (below), so the comparison is like for like against the committed `bench_papyrus.json`.

	bitsandbytes	**vLLM + AWQ**
Detection recall	35/40 = **87.5%**	27/40 = **67.5%**
Findings produced	109	69
Severity agreement	62.9%	59.3%
Extraction succeeded	13/13	11/13
Mean latency	429s	**144.8s**

The speed was real and reproducible: 3.0x. The quality loss was also real, concentrated rather than diffuse (`nda_kestrel` 5/5 to 2/5, `vendor_larkspur` 5/5 to 3/5), and it survived excluding the contracts whose extraction failed. `vendor_brightquay.pdf` additionally stopped extracting at all, stranding five planted defects.

Options. Keep vLLM and restate the benchmark numbers downward; keep it and try to recover recall through prompt work; roll back.

Decision. Roll back. Recall is the product in contract review, and 3.0x speed does not buy back twenty points of it. A slow demonstration is survivable; one that misses a fifth of the planted defects is not. The lane returns to bitsandbytes and both routers return to the client-side lock.

Consequences. Per-contract latency goes back to roughly seven minutes, which the demonstration must be planned around: warm the lane first and prefer the recorded fallback. Rolling back also restores the validity of every benchmark number already published in both systems' documents, which removes the contradiction that had opened between the model cards and the benchmark reports.

One change from the episode is deliberately KEPT, because it is a real defect independent of the swap: `_parse` now falls back to a tolerant read when the model emits Python literals (`True`, `False`, `None`) inside otherwise valid JSON. Strict JSON rejected the entire reply, so a 7kB template-inspector answer containing real findings was discarded over one capital letter, and `temperature=0` meant the retry reproduced the failure exactly rather than recovering. Both attempts failed identically and the inspector reported nothing. The fallback is string-aware, so a finding whose text merely mentions the word True is untouched. Applied to both this system and the governance sibling, which share the parser.

07. Configuration and Secrets

Version 1, 2026-07-28.

1. Rules

- Real values live only in a git-ignored `.env` at the repository root. The repository carries `.env.example` with blank placeholders.
- The lane URL and token are read when the model router is imported, so the API refuses to start without them.
- This system has **no cloud model key at all**, by design. If one appears in the configuration, something has gone wrong.

2. Variables

Name	Required	Meaning
`DATABASE_URL`	yes	Postgres connection string, port 5433, user `legal`, database `contracts`. Use `127.0.0.1` rather than `localhost` to force IPv4
`AUTH_SECRET`	yes	Signs the session cookie. The application refuses to start without it. Changing it signs everyone out
`PRIVATE_LANE_URL`	yes	HTTPS endpoint of the self-hosted model
`PRIVATE_LANE_TOKEN`	yes	Shared secret that endpoint requires
`BRAND_NAME`, `BRAND_TAGLINE`	no	Branding shown in the interface
`CORS_ORIGINS`	no	Comma-separated origins the browser may call the API from, exact origin and no trailing slash. Defaults to `http://localhost:3000`
`COOKIE_SECURE`, `COOKIE_SAMESITE`	no	Default to `false` and `lax`, which is what local development wants. Deployed, with the front end and the API on different domains, they must be `true` and `none` or the session cookie is silently dropped and login appears to do nothing

The same lane endpoint and token are shared with the sibling governance system. Both projects point at one deployment, which is why a blank value in one repository is a common and confusing failure: the review dies before the GPU is ever reached, with a message about an invalid address.

3. Where secrets live

Secret	Home
Lane token	The `.env` file on each machine, and a platform secret on the GPU service side
GPU platform credentials	The platform's own token file in the user profile, on the personal machine only

Secret	Home
Database password	The `.env` file, and it must match the compose file
Session secret	The `.env` file

4. Ports

Service	Address
Web app	http://localhost:3000
API	http://localhost:8000, interactive docs at `/docs`
Weaviate	http://localhost:8081, gRPC 50052
Postgres	localhost:5433

These deliberately avoid the ticket-triage system's ports for the data stores, so both stacks can run at once. The application ports are shared, so only one backend and one web app run at a time.

5. Common configuration mistakes

Symptom	Cause
The API will not start	A lane URL or token, or the session secret, is missing
A review fails immediately with an invalid address	The lane variables are present but blank
Database authentication failed	The connection string disagrees with the compose credentials
Precedent search returns errors, or seeding hangs	A local proxy is intercepting the vector database's gRPC port; exclude localhost
The vector collection does not exist	The precedent seed has never been run on this machine, or the volume was wiped

6. Spend control

Every model call wakes a rented GPU, and the lane bills per warm window rather than per token. A hard spend cap is set on the platform. Measurement runs use a single-contract switch first as a cost fence, then the full corpus in one warm window.

08. Runbook

Version 1, 2026-07-28. First-time installation is in the repository `README.md`; this is for running, recovering and diagnosing.

1. Daily start

```
docker compose up -d                    # Postgres 5433 + Weaviate 8081
uv run uvicorn api:app --reload         # API on :8000
cd frontend && npm run dev              # web app on :3000
```

Behind a TLS-intercepting proxy, set the exclusions **in the same shell** as the backend and as any seed script, or the vector database calls hang and then fail:

```
export NO_PROXY=127.0.0.1,localhost no_grpc_proxy=127.0.0.1,localhost
```

2. Seeding

Command	Effect
`uv run python seed_users.py`	Creates the administrator and lawyer accounts
`uv run python seed_precedent.py`	Fills the precedent cabinet with eight fictional prior reviews. Idempotent and tagged, so re-running never destroys a real filed review

Both are needed on a fresh machine, and again after any `docker compose down -v`.

3. Health checks

Check	How	Healthy answer
API alive	`GET http://localhost:8000/`	Status ok
Sign-in	The login page with a seeded account	Reaches the docket
Model lane warm	A short probe prompt	A short answer in about a minute from cold
Precedent cabinet	Upload and finish any contract, then search	Results, not an error observation

4. Running a review

1. Sign in and drop a `.docx` on the docket.
2. Watch the row narrate: reading, extracting, inspecting, negotiating, summarising, done.
3. Open the review, work down the flagged clauses, and accept, reject or edit each one.

4. Finish the review. It locks and is filed as precedent.
5. Download the corrected document if you need it.

Expect a full review to take several minutes: the four inspectors queue on a single GPU by design.

5. Incidents

The review stops at extraction

Fewer than three clauses were extracted, so the pipeline refused to inspect a document it could not read. Check the file: a scanned PDF has no extractable text. Re-upload as `.docx` if you have one.

One inspector reports failed, the others are fine

Expected behaviour rather than a crash: the review continues with the remaining inspectors and the failure is visible in the report. If it repeats at the identical character on both attempts, that is the fingerprint of truncated model output rather than randomness.

Every model call fails with an invalid address

The lane variables are blank in the environment file. This costs nothing because the run dies before the GPU is reached, but it looks alarming.

The whole review hangs, then times out

The lane is cold, or a container swap happened mid-run. The guard gives each node a bounded wall clock and one retry. Warm the lane and try again rather than retrying cold repeatedly, because each cold wake costs money.

Precedent search returns an error observation to the agents

The vector collection does not exist on this machine, or the proxy is intercepting its port. Run the precedent seed, with the localhost exclusions set.

Finishing a review fails

Either a flagged clause has no decision yet, or the review is already finished. Both are deliberate conflicts, not bugs.

6. Before a demo

1. Warm the model lane about ten minutes ahead. A cold first call takes roughly a minute; a full contract takes several minutes.
2. Have a finished review already in the docket as a fallback, so the walkthrough does not depend on a live run.
3. Follow `demo-script.md` in this folder.

7. Cost discipline

Every wake of the GPU lane costs real money and the free credit is finite. Never run the benchmark contract by contract; use the single-contract switch as a cost fence, then run the whole corpus in one warm window.

09. Test Plan and Report

Version 1, 2026-07-28.

1. Strategy

Layer	Cost	What it proves
Automated tests (`tests/`)	free, zero model calls	Shapes, rules, gates, storage, export, and the audit chain
Benchmark runs (`bench.py`)	GPU money	Whether the pipeline actually finds planted defects. See 10-benchmark-report.md

The automated layer drives the application without starting it, with the model replaced by a fake that recognises each agent by the opening phrase of its prompt. That is why the inspector prompts' opening role phrases must not be reworded casually: the fake model identifies agents by them.

2. Running them

```
uv run pytest -q
```

All 103 are collected with or without a database. The eleven storage tests talk to a real Postgres and skip themselves individually when there is none, so a run with the containers down reports **97 passed, 6 skipped** rather than pretending to be complete.

3. Coverage

103 tests across ten files, green in a few seconds, no model calls and no GPU spend.

File	Tests	Covers
`test_api.py`	17	The endpoints: decision verdicts and their status codes, the finish gates, which text becomes the final wording, and that a precedent-store outage cannot block a lawyer signing off
`test_agents_base.py`	14	The reasoning loop: the step ceiling, duplicate-call blocking, unknown tool handling, and the JSON parser including the first-complete-object behaviour
`test_graph.py`	14	Node guards, the too-few-clauses stop, the fan-in reducer, and inspector status roll-up
`test_agents.py`	12	Agent output shapes against a fake model, finding stamping, and inspector context
`test_state.py`	12	Finding validation and the risk roll-up, including the zero-findings case

File	Tests	Covers
`test_store.py`	11	Postgres round trips, including clause decisions against a real database with its own cleanup
`test_intake.py`	11	File to text for each supported format
`test_audit.py`	6	The hash chain and its verifier
`test_export.py`	4	Document rewriting, including clauses that cannot be located
`test_vocabulary.py`	2	That no vocabulary from the sibling ticket system leaked into this one, with a small allowlist for legitimate legal English

Last recorded run: 103 collected, 97 passed and 6 skipped with the database down; 103 passed with it up.

A defect worth recording, because it is the kind that hides: `test_api.py` guards its import of the application and skipped the whole module when that import failed. `store.init_db()` used to run when `api.py` was merely imported, so on any machine without Postgres, including the CI runner, **those seventeen endpoint tests silently vanished and the suite still reported green**. `init_db()` now runs in the startup handler instead of at import, exactly as the sibling governance system already did it, and the seventeen are back in every run.

4. What testing has caught

- A helper that stamped findings was missing its return, so **every finding from all four inspectors would have been silently dropped**, with no error and no crash.
- Path resolution against the working directory, which would have quietly told all four inspectors that the rules pack and the templates did not exist, if the server were started from anywhere but the repository root.
- A parser that broke deterministically whenever the model emitted a second JSON object.
- A duplicated paste that left the whole API unable to import.

The lesson recorded from those: after any multi-part hand-paste, compile the package before doing anything else.

5. Known gaps

Gap	Why it matters
No end-to-end test through a real model	Only paid benchmark runs exercise the whole nine-node path
No test asserting contract text never leaves for a third party	The claim rests on there being no cloud client in the codebase, which is strong but is not a test
No frontend tests	The review interface is verified by hand
Attribution and severity are measured only by the benchmark	There is no unit-level notion of the right inspector for a defect

6. Test data

Thirteen synthetic contracts with **planted defects and known answers**, which is what makes recall measurable here in a way it is not in the sibling ticket system. Precedent seeds are deliberately built from different counterparties so they cannot leak answers into the benchmark.

10. Benchmark Report

Version 1, 2026-07-28. Raw results are committed as `bench_papyrus.json`.

1. Method

Thirteen synthetic contracts, each carrying **planted defects with known answers**, plus three contracts with a required clause deliberately removed. `bench.py` runs each contract through the full nine-node pipeline on the live model lane and scores:

Measure	Definition
Recall	Planted defects the system found, as a share of those planted
Attribution	Of those found, the share caught by the inspector whose beat it is
Severity	Of those found, the share given the expected severity
Unplanted rate	Findings raised that were not planted
Missing-clause recall	Deliberately removed clauses that were reported as absent
Extraction	Contracts whose clauses were read successfully
Latency	Wall clock per contract

2. Results, full corpus

Measure	Result
Contracts	13
Errors	**0**
Extraction succeeded	13 of 13
Planted defects found	**35 of 40, recall 87.5%**
Correct inspector	71.4%
Correct severity	62.9%
Missing clauses caught	**3 of 3**
Total findings raised	109
Unplanted findings	25, which is 22.9%
Mean latency	429 seconds per contract
Total run	5,577 seconds

An earlier single-contract probe on the same lane returned recall 5 of 5 with zero unplanted noise, in 490 seconds.

3. What these numbers mean

- **Recall is the number that matters and it is high.** The system finds roughly seven of every eight planted defects, and caught every deliberately removed clause. For a review assistant whose output a lawyer checks clause by clause, recall is worth more than precision.
- **Attribution is mediocre and that is acceptable.** Nearly three in ten found defects were caught by a different inspector than the one whose beat it is. The defect is still surfaced with full evidence, so the lawyer sees it. It matters only for explaining which agent does what.
- **Severity agreement is the weakest measure**, at roughly three in five. Severity drives the risk roll-up, so this is the number to improve first if quality work resumes.
- **About a fifth of findings were unplanted.** Some are genuine issues the corpus author did not plant, and some are noise. Without a second reviewer they cannot be separated, so the figure is reported raw rather than adjusted.
- **Seven minutes per contract** is the cost of four inspectors queueing on a single GPU container, which is a deliberate cost decision, not a performance bug.

4. Limits, stated plainly

- Thirteen contracts is a batch, not a statistical sample. No confidence intervals are claimed.
- The corpus is synthetic and written by the same person who built the system, which is the strongest bias in these numbers.
- The precedent cabinet is deliberately seeded from different counterparties, so no inspector can retrieve a planted answer rather than finding it. This is why the recall figure means something.
- Latency assumes a warm lane; a cold start adds about a minute.
- One low-severity planted defect is known to be missed consistently and was deliberately not chased, to avoid tuning the prompts to one contract.

5. Earlier measurement, for contrast

Before the prompt work, the same pipeline on a smaller locally hosted model scored 2 of 5 on one contract. Four targeted prompt fixes, none of them model changes, took that to 4 of 5 on the same contract and 5 of 5 on an unseen one with zero unplanted noise. The fixes were: a three-step working method telling an inspector to sweep every clause to the last; teaching the financial inspector that penalties are money terms even without a payment schedule; making the loop carry issues noticed mid-thought into the final findings; and correcting invented tool names instead of failing.

That sequence is the useful story: the gains came from telling the agents how to work, not from a larger model.

5a. The vLLM + AWQ trial, and why the numbers above still stand

On 2026-07-28 the lane was swapped to vLLM serving the official AWQ checkpoint, for speed. On 2026-07-29 the **full corpus was rerun on that stack** and the swap was rolled back. Both result files are in the

repository, so the comparison can be checked rather than taken on trust: `bench_papyrus.json` is the shipped stack, `bench_papyrus_vllm_13.json` is the trial.

	bitsandbytes (shipped)	**vLLM + AWQ (rejected)**
Detection recall	**87.5%**	67.5%
Findings produced	109	69
Severity agreement	62.9%	59.3%
Extraction succeeded	13/13	11/13
Mean latency	429s	**144.8s**

Three times faster, twenty points less recall. The decision and its reasoning are ADR-012.

Two things are worth recording about how this was measured. The first rerun attempt was **abandoned partway** because the log showed inspectors failing to parse their own output; continuing would have measured a bug rather than a model. The cause was the model emitting Python literals inside its JSON, which made strict parsing discard entire inspector answers. That was fixed first, and only then was the comparison run, so the 67.5% is the model's real performance and not an artifact.

The second: a single contract had suggested this same conclusion a day earlier, at 3/5 against 5/5. That signal was correct, but it was **not trustworthy at the time** and was rightly not acted on, because one contract cannot separate quantisation damage from ordinary run-to-run variation. Thirteen can.

6. Reproducing

```
uv run python bench.py --only kestrel     # one contract, the cost fence
uv run python bench.py                    # the full corpus, one warm window
```

Each run costs real GPU money. Warm the lane first and never run the corpus one contract at a time.

11. Non-Functional Requirements

Version 1, 2026-07-28.

1. Performance

Property	Target	Measured
Full contract review	minutes, unattended	mean 429 seconds over 13 contracts, warm lane
First call after the lane idles	under two minutes	roughly 60 to 90 seconds
Per-node ceiling	bounded, with one retry	1200 seconds per guarded node
Whole-contract ceiling in the benchmark	bounded	2700 seconds
Upload response	immediate	The file is stored and an id returned before any model runs

The dominant cost is deliberate: four inspectors queue on one GPU container rather than running truly in parallel, because letting the platform start a second container doubled the bill and broke mid-run.

2. Scale

Single concurrent user, one contract at a time, a corpus in the low tens. Contracts of about nine to fourteen clauses are the tested size. Nothing here is designed for a firm's real caseload.

3. Availability

No availability target and no failover. The narrower guarantee that is met: **a failure never loses a review and never hides itself.** A node that fails records the failure in the state, the other inspectors still produce findings, and the review reaches the lawyer with the gap visible. A document that could not be read stops with an explicit extraction failure rather than being inspected on empty text.

4. Privacy

Requirement	Status
Contract text never reaches a third-party model	Met by construction: one lane, and no cloud client anywhere in the codebase
Embeddings computed locally	Met
Original documents stored locally	Met, kept as bytes in the local database
No open signup	Met, an administrator creates every account

This is a stronger position than the sibling ticket system, which has a cloud lane for non-sensitive work. Here the question does not arise.

5. Auditability

Every step appends to a hash chain stored with the contract and verified when read through the audit endpoint, which reports the first broken index. This has been tested against a row forged directly in the database and the break was reported at the exact index. The chain proves tampering; it does not prevent it.

6. Quality bar

A finding must carry all nine required fields or it is discarded before display. That is a deliberate quality-over-quantity choice: the measured recall of 87.5 percent is achieved **after** dropping incomplete findings, not before.

7. Cost

The GPU lane bills per warm window. A full benchmark of thirteen contracts is roughly an hour and a half of GPU time and is the single largest cost in the project. Controls: a hard platform spend cap, a single-container lane, a single-contract switch as a cost fence, and batching every paid check into one warm window.

8. Maintainability

The system is a fork of the ticket-triage skeleton, and several modules are near-identical to their siblings. That duplication is measured, and the decision to replace it with a shared package after the deadline is recorded in the planning repository. Until then, a fix in a shared-looking module should be checked in all three repositories.

12. Security Review

Version 1, 2026-07-28. A threat model for a single-machine demonstration system holding confidential documents.

1. Assets

Contract text and the original uploaded files; the findings and proposals, which reveal a negotiating position; account credentials; the model lane token; the audit chain's integrity.

2. Trust boundaries

Boundary	Other side
Browser to API	Anyone who can reach the port
API to the model lane	An internet-reachable GPU endpoint
API to data stores	Local Docker containers

There is deliberately **no third-party model provider** in this list.

3. Threats and controls

#	Threat	Control today	Residual risk
1	An outsider creates an account and reads contracts	**No open signup.** Only an administrator creates accounts	An administrator account is a single point of trust
1a	An outsider claims the founding administrator through the first-run setup route	`POST /auth/bootstrap` is unauthenticated by necessity, but it counts the accounts first and refuses with 403 the moment any exists. On a seeded or running system it is already closed	**A system that is deployed but never seeded is claimable by the first visitor.** Whoever installs it must complete setup before publishing the URL
2	Password guessing	Hashed passwords	**No rate limiting or lockout**
3	Session theft	Signed cookie with a secret from the environment	The cookie is not marked secure, because the demonstration runs over plain HTTP locally
4	Contract text leaks to a model provider	There is no cloud client in the codebase and only one lane exists	The lane is rented infrastructure, so text does leave the machine to a self-hosted endpoint over an authenticated connection
5	The lane token is stolen and the GPU budget is spent	Shared-secret token and a hard platform spend cap	The endpoint is internet reachable; the cap is the real backstop

#	Threat	Control today	Residual risk
6	A lawyer sees another firm's contracts	Every account sees every contract in this instance	**No per-matter access control.** Acceptable for a demonstration, unacceptable for real use
7	The audit trail is edited to hide a decision	Hash chain, verified on read, tested against a forged database row	Tamper evident, not tamper proof; nothing external notarises it
8	A malicious document exploits the parser	Only two formats are accepted, and text extraction runs in-process	No sandboxing of document parsing
9	Prompt injection inside a contract, aiming to suppress a finding or plant wording	Every finding must carry a quote of the offending wording, and a lawyer decides every flagged clause	**Not systematically tested.** A crafted clause instructing the reviewer is a plausible attack on this exact product
10	An exported document contains wording the lawyer never approved	Export only writes clauses whose decision is recorded, and unlocatable clauses are named rather than guessed	Matching is by wording, so an unusual document could match the wrong paragraph
11	Secrets committed to the repository	Only blank placeholders are committed	Nothing scans commits

4. What would have to change before real client documents

In priority order:

1. Per-matter access control, so an account reaches only its own files.
2. Rate limiting and lockout on sign-in.
3. HTTPS with a secure session cookie.
4. Encryption at rest for the database, since the original documents are stored as bytes.
5. A tested position on prompt injection inside contract text.
6. A retention and deletion workflow, including removal from the precedent cabinet.

5. Deliberate non-goals

No penetration test, no dependency vulnerability scanning, no formal access model beyond two roles. This is a demonstration system on synthetic contracts, and nothing here should be read as production readiness for legal work.

13. Privacy and Data Handling

Version 1, 2026-07-28.

1. Position

Every contract in this system is synthetic. The design nevertheless assumes real confidential documents, because that is the only way the privacy claim is worth making: **contract text never reaches a third-party model provider, and the codebase contains no client that could send it to one.**

2. What data the system holds

Data	Where
The original uploaded document, as bytes	The contracts table, kept so export can rewrite it
Extracted plain text and the clause list	Inside the stored state
Findings, proposals, negotiating positions	Inside the stored state
The lawyer's decisions and final wording	Inside the stored state
A summary of each finished review	The precedent cabinet, retrievable for later contracts
Account details	The accounts table, passwords hashed

Findings and negotiation points deserve particular care: an ask, a fallback and a walk-away position are more sensitive than the contract itself, because they reveal what the client will settle for.

3. Where data flows

Flow	Destination
Extraction, inspection, negotiation, summary	The self-hosted model endpoint on rented GPU infrastructure, over an authenticated connection
Embeddings for precedent search	Computed on the local machine; no text is sent out to be indexed
Storage	Local Postgres and local Weaviate, both in Docker
Export	Produced locally, downloaded by the lawyer

Nothing is sent to a counterparty by the system. The corrected document is a download; a human decides what leaves.

The honest qualification: the model lane is self-hosted but not on premises. The weights and runtime are ours and no provider trains on the traffic, but contract text does travel to that endpoint.

4. Precedent cabinet and confidentiality

Finishing a review files a summary of it into a shared searchable cabinet, and a later contract, potentially for a different client, can retrieve it. On synthetic data that is the intended learning loop. With real client work it would be a **conflict and confidentiality problem**, and would need at minimum: anonymisation of parties before filing, and a matter or client boundary on retrieval. That is the single largest change this system would need before real use, and it is a design change rather than a bug.

5. Retention

Nothing is deleted. Contracts, their original bytes, their state and their precedent entries are kept indefinitely. There is no retention schedule, no deletion endpoint, and no way to remove a review from the precedent cabinet. Acceptable for synthetic material only.

6. Access

Two roles. A lawyer reaches every contract in the instance; an administrator additionally manages accounts. There is no per-matter or per-client boundary, which is called out in 12-security-review.md as the first control to add.

7. Gaps to close before real documents

1. A matter boundary on both contracts and precedent retrieval.
2. Anonymisation of parties before a review is filed as precedent.
3. Retention and deletion, including from the vector store.
4. Encryption at rest, since the original documents are stored as bytes.
5. A record of which model version reviewed which contract, retained for audit alongside the hash chain.

14. Model Card

Version 1, 2026-07-28.

1. The model

Property	Value
Model	Qwen2.5-14B-Instruct, 4-bit quantised with bitsandbytes
Serving	One-at-a-time transformers generation on a single GPU container. An AWQ + vLLM lane ran from 2026-07-28 to 2026-07-29 and was rolled back: 3.0x faster but recall fell from 87.5% to 67.5% over the same 13 contracts (ADR-014)
Host	Serverless GPU (A10G), single container, five-minute warm window
Lanes	**One.** No cloud model, no tier switch, no fallback
Embeddings	A small local model, computed on the machine
Sampling	Greedy, so runs are as reproducible as the model allows
Token ceiling per call	Generous, because a finding carries nine fields and several findings in one answer will otherwise be truncated

Why this model and not a bigger one. An earlier, larger model (Qwen3-30B-A3B) was tried first and abandoned on 2026-07-24. It took four to six minutes to load, which exceeded every timeout in the stack and made a cold call impossible to answer, and it needed cards costing roughly twice as much. The 14B model loads in about a minute, which is what makes a cold demo call survivable. The comment at the top of `modal_lane/llm_service.py` records the swap.

2. What the model is asked to do

Agent	Output contract
Extraction	Clause list with numbers, headings, wording and types, plus parties, contract type and dates
Compliance	Findings against the firm's rules pack
Risk	Findings on liability, indemnity, intellectual property, renewal, restraint
Template	Deviations from the standard, and required clauses that are absent
Financial	Findings on payment terms, increases, interest, penalties, totals
Negotiation	One replacement clause per flagged clause, with ask, fallback and walk-away
Summary	Executive text and counts

Each inspector runs as a reasoning loop with up to six steps and three read-only tools.

3. Measured behaviour

From the thirteen-contract benchmark: recall 87.5 percent of planted defects, all three deliberately removed clauses caught, correct inspector 71 percent, correct severity 63 percent, roughly one finding in five unplanted, zero errors, mean 429 seconds per contract. Full detail and caveats in 10-benchmark-report.md.

4. Known failure modes

Failure	How it shows	What contains it
Truncated output	A parse failure at the identical character on both attempts	A generous token ceiling; the fingerprint is documented so it is not mistaken for randomness
A second JSON object after the answer	Parse failure that repeats deterministically	The parser takes the first complete object
An inspector burns its loop repeating one tool, then concludes nothing is wrong	A confident empty result on a contract that clearly has issues	An explicit three-step working method in the prompt: use the tool, then sweep every clause to the last, then report every issue including ones noticed mid-thought
An issue spotted in an early thought is forgotten by the final answer	A finding that appears in the reasoning but not the output	The loop's finish instruction requires carrying earlier observations into the findings
An invented tool name	A wasted step	Unknown names are corrected with the real options
Attribution drift	A defect caught by the wrong inspector	Accepted and measured; the finding still reaches the lawyer
Severity disagreement	Risk roll-up skewed	Known weakest measure, at roughly three in five
Cold container	The first call after idle takes about a minute	Warm the lane before a demonstration

5. What is not measured

Precision cannot be separated from genuine discovery: a fifth of findings were not planted, and some of those are real issues the corpus author missed. There is no second reviewer, so that figure is reported raw. There is also no measure of whether a proposed replacement clause is legally sound; a lawyer decides that, which is the entire point of the clause-by-clause gate.

6. Appropriate and inappropriate use

Appropriate: a first pass over a commercial contract, producing evidence-carrying findings and draft replacement wording that a lawyer reviews clause by clause.

Inappropriate: any use where the output is acted on without a qualified human reading it; jurisdiction-specific advice; execution or signature workflows; real client documents before the confidentiality gaps in

13-privacy-and-data-handling.md are closed.

7. Human oversight

Nothing is finished without a decision on every flagged clause. The final wording of every clause is either the original, the proposal a lawyer accepted, or the lawyer's own edit. Export writes only what was decided.

15. Risk Register

Version 1, 2026-07-28. Likelihood and impact are low, medium or high.

Open risks

#	Risk	Likelihood	Impact	Mitigation
R-1	A live demonstration runs on a cold lane, so a review takes far longer than the audience will wait	Medium	High	Warm the lane ten minutes ahead, keep a finished review in the docket, and follow the demo script
R-2	GPU spend exhausts the credit before the work is finished	Medium	High	Hard platform cap, single-container lane, a one-contract cost fence, and batching every paid check into one warm window
R-3	Severity judgement is only about three in five correct, and severity drives the risk roll-up	High	Medium	Reported openly in the benchmark. The first quality work to resume if time allows
R-4	A fifth of findings are unplanted, and some may be noise a lawyer must wade through	Medium	Medium	Findings must be complete and carry evidence, so each is quick to dismiss. Not separable without a second reviewer
R-5	Prompt injection inside a contract steers or suppresses a finding	Medium	Medium	Evidence quotes and the clause-by-clause human gate. Not systematically tested, and this product is an obvious target for it
R-6	Export matches the wrong paragraph in an unusual document	Low	Medium	Matching is on the clause's original wording, and anything unlocatable is named rather than guessed
R-7	The precedent cabinet mixes clients, which would be a real confidentiality problem outside a demonstration	Low here, High in production	High	Documented as the largest design change needed before real use
R-8	Duplicated modules mean a bug fixed here is left unfixed in the sibling systems	Medium	Medium	The duplication is measured and the shared-package plan is written. Two such bugs have already occurred
R-9	The README describes an older model than the code runs	Certain, already true	Low	Corrected in the model card; the README line should be updated
R-10	No per-matter access control	Low here, High in production	High	Named as the first control to add before real client documents

Closed risks

#	Risk	How it closed
C-1	The model took four to six minutes to load, so a cold call could never answer	Timeouts raised in the right order, then a smaller model that loads in about a minute
C-2	Parallel inspectors woke a second billed GPU whose model was still loading, killing runs mid-way	Single-container lane, a client-side lock, and one retry on a stray server error
C-3	Every finding from all four inspectors would have been silently dropped by a helper missing its return	Caught by reading the code after a hand-paste; the standing rule is now to compile after any multi-part paste
C-4	All four inspectors could have been silently told the rules pack did not exist, if the server were started from another directory	Paths anchored to the code rather than the working directory, and verified from a foreign directory
C-5	A second JSON object in the model's output broke parsing deterministically	The parser takes the first complete object
C-6	The precedent cabinet was empty on a fresh machine, so early contracts got nothing from it	Eight seeded fictional reviews, deliberately not drawn from the benchmark corpus
C-7	The audit chain was built but never exposed or verified	Exposed through an endpoint, verified on read, and tested against a forged database row

16. Traceability Matrix

Version 1, 2026-07-28. Each requirement from 01-requirements.md, the code that satisfies it, and the test or measurement that proves it.

Functional

Req	Verdict	Code	Proof
FR-1 upload and normalise, identify type, parties, dates	MET	`POST /contracts`, `app/intake.py`, extraction agent	`test_intake.py` (11 tests); benchmark extraction 13 of 13
FR-2 clause splitting with types	MET	`extraction_agent` in `app/agents.py`, vocabulary in `app/state.py`	`test_agents.py`; benchmark
FR-3 four-way inspection	MET	Four inspector agents, parallel step in `app/graph.py`	`test_agents.py`, `test_graph.py`; benchmark recall 87.5%
FR-4 missing clauses reported	MET	Template inspector, `missing_clauses` in the state	Benchmark: 3 of 3 caught
FR-5 findings complete or discarded	MET	`valid_finding()` in `app/state.py`, `_stamp` in `app/agents.py`	`test_state.py` (12 tests)
FR-6 replacement wording with ask, fallback, walk-away	MET	`negotiation_agent`	`test_agents.py`; benchmark
FR-7 risk roll-up and executive summary	MET	`risk_rollup()` in `app/state.py`, `summary_agent`	`test_state.py`, including the zero-findings case
FR-8 clause-by-clause accept, reject, edit	MET	`POST /contracts/{id}/clauses/{clause_id}/decision`	`test_api.py`: verdicts, status codes, which text becomes final
FR-9 precedent retrieval	MET	`app/precedent.py`, `precedent_search` tool	`test_api.py` proves an outage cannot block sign-off; seeding proven live

Req	Verdict	Code	Proof
FR-10 tamper-evident audit, readable	MET	`app/audit.py`, `GET /contracts/{id}/audit`	`test_audit.py` (6 tests); verified live against a forged database row
FR-11 export the corrected document	MET	`app/export.py`, `GET /contracts/{id}/export`	`test_export.py` (4 tests) including unlocatable clauses
FR-12 administrator-created accounts, no open signup	MET	`app/users.py`, the users endpoints	`test_api.py` role gates

Non-functional

Req	Verdict	Evidence
NFR-1 no third-party model	MET	One lane in `app/router.py`; no cloud client exists in the codebase. **Not covered by a test**, which is the one gap worth closing
NFR-2 unattended review completes	MET	Benchmark mean 429 seconds, 0 errors over 13 contracts
NFR-3 a stuck agent cannot hang the review	MET	`guarded()` in `app/graph.py`; `test_graph.py` covers node guards
NFR-4 parallel inspectors do not multiply cost	MET	Single-container lane plus the lock in `app/router.py`
NFR-5 no half-formed finding reaches a lawyer	MET	`valid_finding()`, with drops counted at fan-in; `test_state.py`, `test_graph.py`
NFR-6 tampering detectable	MET	`test_audit.py` plus the live forged-row check
NFR-7 synthetic data, secrets outside the repository	MET	13 synthetic contracts with manifests; ignored environment file
NFR-8 reuse of the skeleton	MET	Forked from the ticket system; duplication measured in the planning repository

Coverage summary

103 automated tests, no model calls, covering every deterministic seam: shapes, validation, the reasoning loop, node guards, the fan-in reducer, storage, export, the audit chain, the endpoint gates, and a check that no vocabulary from the sibling ticket system leaked in.

The two things automated tests do **not** cover are stated plainly: the full nine-node path through a real model, which only the paid benchmark exercises, and an assertion that contract text can never leave for

a third party, which today rests on the absence of any cloud client rather than on a test.

17. User Guide

Version 1, 2026-07-28. For the lawyer using the system, and the administrator running it.

1. Signing in

There is no self-service signup. An administrator creates your account and gives you the address and password. You may sign in with **either your email address or your username**; capitals do not matter. Lawyers land on the docket, administrators land on the people page.

The very first time the system is ever started, there are no accounts at all, so there is nobody to create yours. The sign-in screen notices this and turns itself into a one-time setup form: fill in an email address, a username and a password, and it creates the founding administrator. That administrator then creates everyone else. Once a single account exists this screen never appears again, and the setup route refuses anyone who tries it later.

2. The docket

The docket is the list of contracts. Each row shows the file, its status, its stage while it is being worked on, and its risk level once finished.

Drop a `.docx` (or a `.pdf`) on the docket to start a review. The row appears immediately and narrates what the system is doing:

Stage	Meaning
reading	Turning the file into text
extracting	Splitting it into numbered clauses
inspecting	Four reviewers going through every clause
negotiating	Drafting replacement wording for what was flagged
summarising	Writing the executive summary
done	Ready for you

A full review takes several minutes. That is the four reviewers taking turns on one machine, by design.

If a row says extraction failed, the document could not be read into at least three clauses. A scanned PDF with no text layer is the usual cause. The system refuses to inspect a document it could not read, rather than reviewing an empty page.

3. Reading a review

Opening a contract gives you the clause list on one side and the detail on the other.

For each clause you see the original wording, and where something was flagged:

Part of a finding	What it tells you
The plain line	What the problem actually means, in everyday words
The term	The legal name for it
What is wrong	The specific defect
What to change	The fix being proposed
If ignored	The consequence of leaving it
The quote	The exact wording in the contract that caused it
Severity	High, medium or low, which feeds the contract's risk level

Four reviewers produce these: one checks the firm's rules, one commercial risk, one deviation from your standard template, and one the money terms. A finding that arrives incomplete is thrown away before you see it, so everything on screen is actionable.

Missing clauses are listed too. A required clause that simply is not in the contract is treated as a finding in its own right.

4. Deciding

Every flagged clause needs your decision:

Action	Effect
Accept	The proposed replacement wording becomes the final wording
Reject	The original wording stands
Edit	Your own wording becomes the final wording

You cannot finish a review while any flagged clause is undecided. That is deliberate.

5. Finishing and exporting

Finishing locks the review and files it into the precedent cabinet, so a future contract with a similar term can retrieve what was decided here.

After finishing, download the corrected document. It is your original file with **only the changed clauses rewritten**, so formatting, numbering and everything you did not touch survive untouched. Two limits, both shown rather than hidden: export works on `.docx` only, and if a clause's original wording could not be located in the file, it is named for you to fix by hand.

6. The audit trail

Every step the system took is recorded in a chain where each entry is sealed against the one before it. Open the audit panel to read it. If any past entry were altered, the chain reports exactly where it broke.

7. For administrators

The people page is where an administrator lands after signing in. It creates accounts, changes roles between lawyer and administrator, resets an address, username or password, and deactivates an account. You cannot deactivate your own account. An administrator can create other administrators.

Administrators can also open the docket and read reviews. The docket is shared, not a second copy: an administrator sees the same contracts the lawyers do.

8. Worth knowing

- **The first review after a quiet period is slower.** The model server sleeps and takes about a minute to wake.
- **Contracts never leave for an outside model provider.** There is one self-hosted model and no cloud path in the system at all.
- **Every finding carries its evidence.** If you cannot see why a finding was raised, the quote in it tells you which wording triggered it.
- **The system does not give legal advice.** It surfaces issues and drafts wording. Every decision is yours.

18. Release Notes

Version 1, 2026-07-28. No version tags; 27 commits, from 2026-07-20 to 2026-07-27. Grouped by theme.

2026-07-27, reporting

- Per-contract risk panel and an exportable review report.
- Documentation corrections to the sign-in credentials.

2026-07-26, measurement

- **Full thirteen-contract benchmark committed:** recall 87.5 percent, all three deliberately removed clauses caught, zero errors, mean 429 seconds per contract.

2026-07-25, the document as the deliverable

- **Corrected-document export.** The reviewed contract is returned as the original file with only the changed clauses rewritten, matching each clause's original wording against a moving window of paragraphs. Uploads now keep the original bytes, which is why contracts uploaded before this change must be re-uploaded to export.
- Limits surfaced rather than silent: `.docx` only, and unlocatable clauses named in a response header and in the interface.
- **Fix:** the reasoning loop now takes the first complete JSON object from model output. The previous parse, from the first brace to the last, broke deterministically whenever the model emitted a second object. The identical bug existed in the sibling governance system and was fixed there too.

2026-07-24, making the model lane usable

Three layers of failure, all fixed in one session:

- **Timeouts:** the stack's ceilings were ordered smallest-first, so nothing could ever finish. Raised in the correct order, and the model later swapped for one that loads in about a minute rather than four to six.
- **Truncation:** every reasoning call used a small token ceiling, so a finish message carrying several nine-field findings was cut off mid-string. Raised.
- **Concurrency, the expensive one:** four inspectors fanning out made the platform start a second billed GPU whose model was still loading, so requests queued for minutes and died. The lane is now pinned to one container, calls are serialised client-side, and a stray server error is retried once.
- **Recall tuning, free, on a local model:** one contract went from 2 of 5 to 4 of 5, and an unseen one to 5 of 5 with zero unplanted noise. Four prompt fixes did it: a three-step working method,

teaching the financial inspector that penalties are money terms, carrying mid-thought observations into the final findings, and correcting invented tool names.

2026-07-22, testing and the interface

- **The lawyer's desk:** docket with live stage narration, the two-pane redline review, and the people administration page.
- **Test suite** over every deterministic seam, no model calls.
- **Audit trail exposed** through an endpoint and a panel.
- **Precedent cabinet seeded** with eight fictional prior reviews, deliberately not drawn from the benchmark corpus so recall still means something.
- **Benchmark harness** scoring against the planted manifests.
- **Fixes:** tool paths anchored to the repository root rather than the working directory, which would otherwise have quietly told all four inspectors that the rules pack did not exist; fan-in now separates an incomplete finding from one naming an unknown clause; and a duplicated paste that left the API unable to import.

2026-07-21, the agents

- Tool registry, precedent store, and the four inspectors.
- Negotiation and summary agents, and the full nine-node graph.
- Lawyer decision endpoints, finish, and precedent filing.

2026-07-20, the fork

- Forked from the ticket-triage skeleton: stripped, authentication swapped to administrator-created accounts with no open signup, and its own database and vector stack on separate ports so both systems can run at once.
- Contract state, storage, and document intake for `.docx` and `.pdf`.
- Data: the template library, the rules pack, and thirteen synthetic contracts with planted-defect manifests.
- Shared reasoning loop and the extraction agent.

Known issues carried forward

- The README still names an earlier, larger model than the code runs.
- A contract with zero findings is reported as low risk, indistinguishable from a slightly untidy one. Pinned by a test so changing it is deliberate.
- One low-severity planted defect is consistently missed, deliberately not chased to avoid tuning prompts to a single contract.
- Severity agreement is the weakest measured dimension at roughly three in five.

19. Handover

Version 1, 2026-07-28.

1. Get it running

Repository `README.md` for installation, 08-runbook.md for daily operation. You need Docker, `uv`, Node, and an environment file. The model endpoint and its token are the only values you cannot invent locally; without them the API refuses to start.

Order: containers up, seed the accounts, seed the precedent cabinet, start the API, start the web app, sign in.

2. Read these first

1. 02-hld.md, the pipeline diagram.
2. 03-lld.md sections 2 to 4: the three shapes, the reasoning loop, and the guards.
3. 06-adr-log.md, especially the single-lane and single-container decisions, which are where the money and the privacy claim live.

3. The five things that will surprise you

1. **Every model call costs money.** The lane wakes a rented GPU and bills a warm window. Never loop the benchmark contract by contract; use the one-contract switch, then the full corpus in one window.
2. **The four inspectors do not really run in parallel.** They fan out in the graph, then queue on one GPU container, because letting the platform start a second one doubled the bill and broke runs mid-way. Seven minutes a contract is that decision, not a performance defect.
3. **The fake model in the tests recognises agents by the opening phrase of their prompts.** Rewording an inspector's opening role line will break tests in a way that looks unrelated.
4. **A finding missing any of its nine fields is discarded silently and counted.** If findings vanish, look at validation before looking at the model.
5. **Two failure fingerprints are worth memorising.** A parse failure at the identical character on both attempts is truncation, not randomness. An immediate invalid-address error means the lane variables are blank, and it costs nothing.

4. Where the important logic lives

Question	File
What counts as a valid finding	`valid_finding()` in `app/state.py`
How findings attach to clauses and risk is rolled up	`fan_in()` in `app/graph.py`

Question	File
How an inspector thinks and uses tools	`app/agents_base.py`
What each inspector looks for	The four prompts in `app/agents.py`
How the model is called, and why it locks	`app/router.py`
How the corrected document is produced	`app/export.py`
What a lawyer's decision does to the final wording	The decision endpoint in `api.py`

5. Open work, in the order worth doing

1. **A test asserting contract text can never reach a third party.** The claim is currently proved by the absence of a cloud client, which is strong but is not a test, and it is the system's headline privacy promise.
2. **Severity agreement**, the weakest measured dimension, and the one that drives the risk roll-up.
3. **Update the README's model line**, which still names the earlier, larger model.
4. **A matter or client boundary** on both contracts and precedent retrieval, which is the first real-world blocker.
5. **Retention and deletion**, including removal from the precedent cabinet.
6. **Extract the shared core** with the sibling systems, per the decision recorded in the planning repository, deliberately scheduled after the current deadline.

6. Operational cautions

- `docker compose down -v` destroys the precedent cabinet and every contract. Re-seed afterwards.
- Changing the session secret signs everyone out.
- The database password in the environment file must match the compose file; Postgres only applies credentials when it first initialises an empty volume.
- After any multi-part hand-paste, compile the package before running anything. Three separate incidents in this repository came from pasted code, and each would have been caught in two seconds.

7. Related repositories

This system was forked from the ticket-triage system and is the parent of the AI-governance system. Several modules are near-identical across all three; the duplication is measured and the shared-package plan is recorded in the planning repository. A fix in a shared-looking module here is worth checking in both siblings, because that has already been necessary twice.